Weather Record Program

Test Plan:

1) Verify that all classes function correctly in isolation and fit in into the existing program without violating restrictions.

2) Scope

- Bst<T> – insert/search/copy/traversals via function-pointer callbacks (const T&).
- Map<K,V> – minimal map wrapper (inserts, contains, get).
- DateRecMap – per-date record map
- MonthYearKey – ordering/fairness for (year, month) keys.
- DateUtils – date comparisons, month names, stream output.

Out of scope (already tested in A1):
Vector, Date, MyTime, WeatherRecord, MonthlyData, Stats, UnitConvert, WindlogAggregator, Menu,RecordParse,FileReader

Test Approach

Unit Testing

- Bst<T>: insert/search; in-order traversal; deep copy; assignment; empty-edge; duplicates.
- Map<K,V>: insertion success/fail; contains; get (const & non-const) referenc.
- DateRecMap: add across multiple dates; get existing/missing; ordering not required; independence per date.
- FileReader: handles multiple files
- MonthYearKey: strict ordering; equality; boundary months/years.

Integration

- BST × DateRecMap: traverse dates (BST) → pull daily vectors (DateRecMap) without crashes.
- Map × MonthYearKey: randomized key insertion → sorted iteration and keyed retrieval.

4) Test Data

4.1 Data

data/data_source_test.txt

MetData-Test-1.csv

MetData-Test-2.csv

MetData-01-2024.csv

4.2 CSVs

MetData-01-2024.csv

```
WAST,S,T,SR
01/01/2024 00:00,4.0,10.0,300
01/01/2024 01:00,6.0,12.0,600
01/01/2024 02:00,8.0,14.0,900
01/01/2024 03:00,10.0,16.0,1200
01/01/2024 04:00,12.0,18.0,1500
```

MetData-Test-1.csv

```
WAST,S,SR,T
15/03/2016 09:00,8,6501,22.5
15/03/2016 10:00,7,7201,23.1
20/04/2016 13:00,7,9201,28.5
15/05/2016 13:00,7,1010,31.2
25/05/2016 14:00,9,5001,20.5
01/06/2016 11:00,12,1100,32.8
19/06/2016 12:00,10,2001,34.2
25/07/2016 14:00,10,1230,36.8
30/07/2016 15:00,7,2341,38.3
15/08/2016 09:00,9,1180,34.5
20/09/2016 14:00,4,9901,30.1
15/11/2016 09:00,5,6501,22.8
```

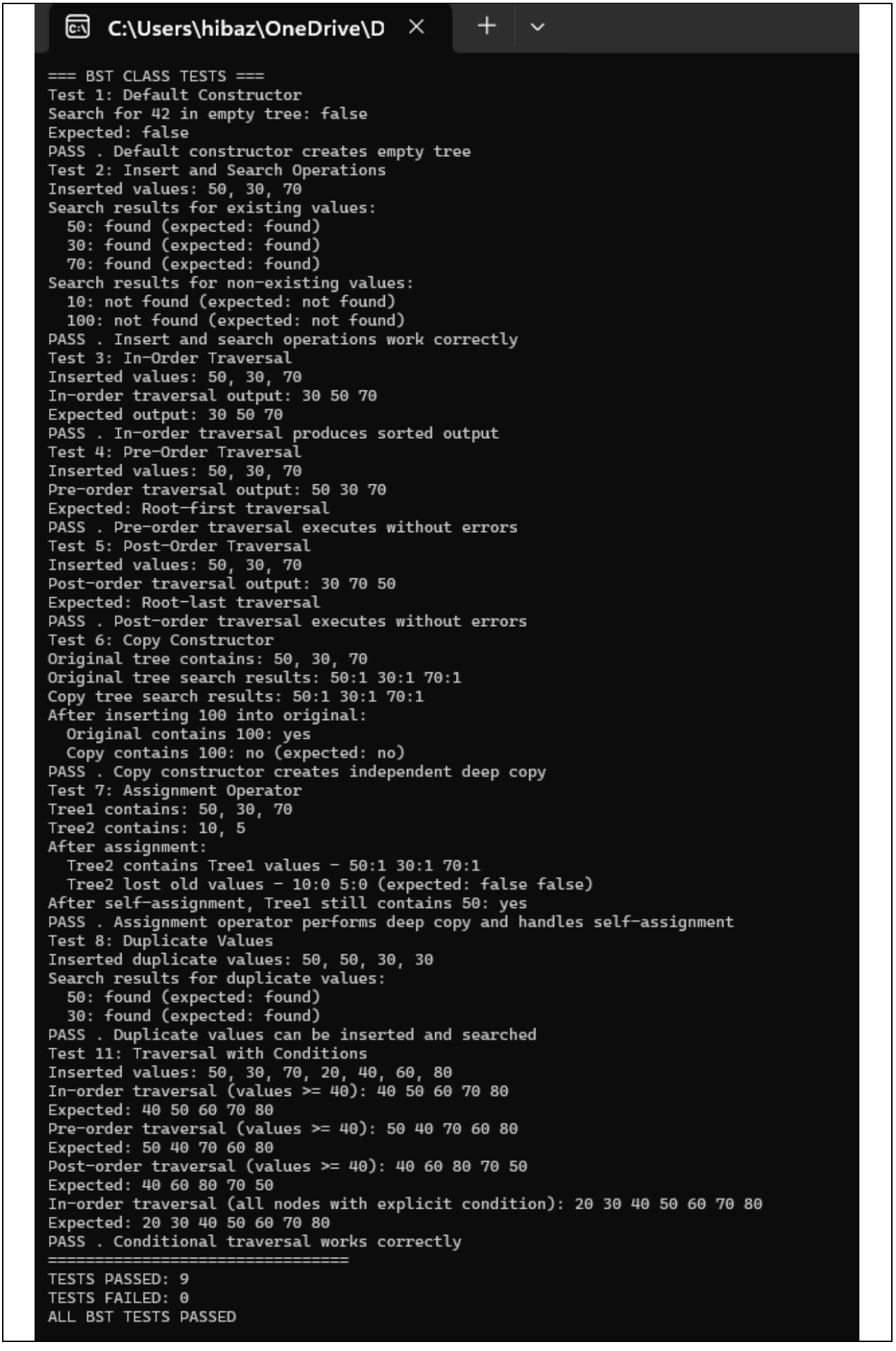
MetData-Test-1.csv

```
WAST,S,SR,T
10/01/2017 09:00,6,480,17.5
10/01/2017 10:00,5,500,18.1
25/01/2017 11:00,4,550,19.2
25/02/2017 12:00,3,570,
15/02/2017 10:00,6,640,22.1
15/03/2017 09:00,8,750,24.5

20/04/2017 11:00,7,950,29.2
05/05/2017 09:00,10,1020,30.5
,,,,,,,,,,
20/06/2017 11:00,9,1200,35.2
10/07/2017 09:00,12,1250,36.5
```

5) Detailed Unit Test Cases

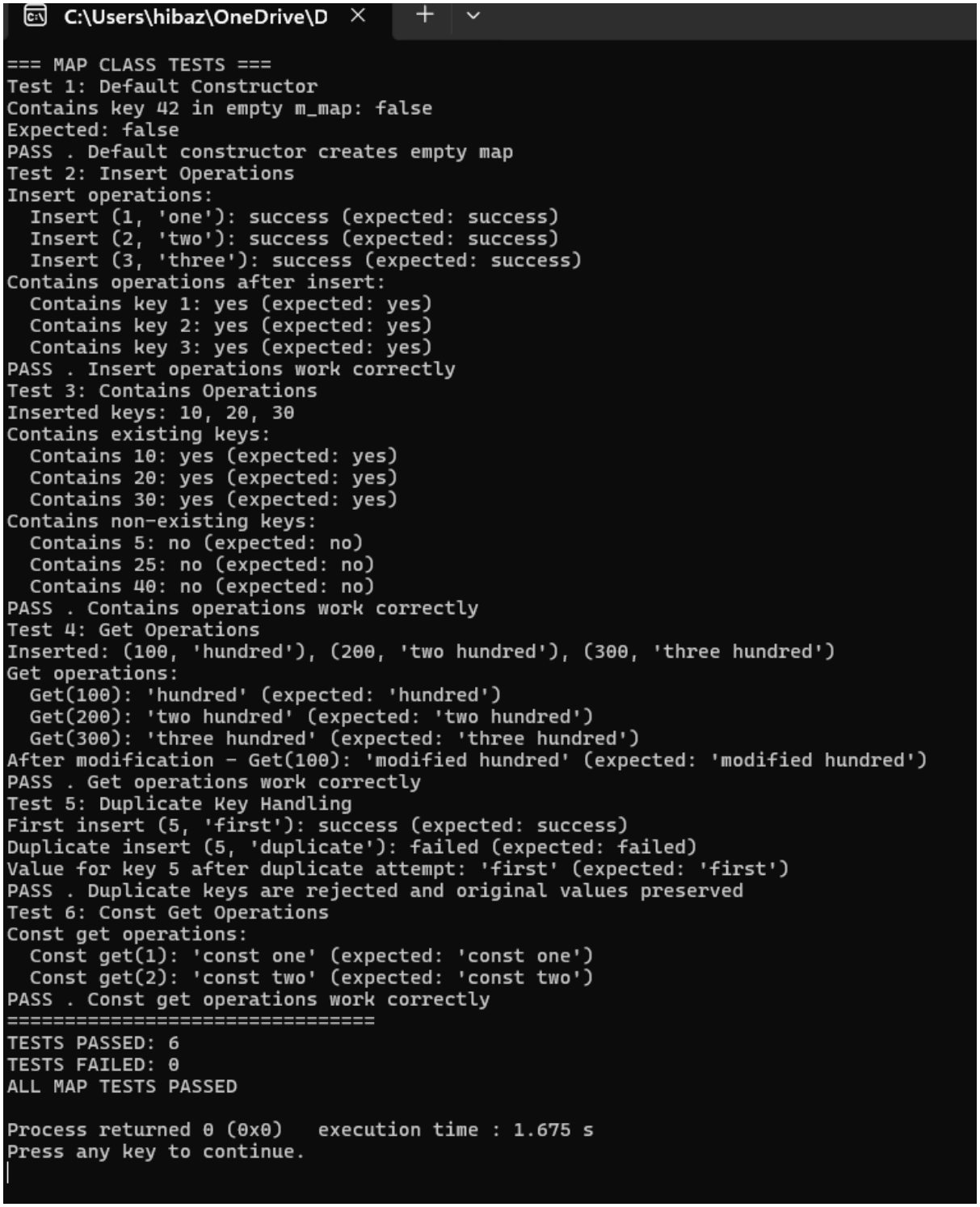
5.1 Bst<T>

Test	Description	Actual Test Data	Expected Output	Pass/fail
1	Default constructor initializes an empty tree	Construct Bst<int> t;	search(42)==false;	Pass
2	insert() builds proper BST with entered values	Insert: 50,30,70	Structure is valid; all inserts succeed	Pass
3	search() finds existing keys	Search 50,30,70	Returns true for each	Pass
4	search() rejects absent keys	Search 10,100	Returns false for each	Pass
5	In-order traversal gives sorted order	In-order callback with nullptr condition	Output: 30,50,70	Pass
6	Pre-/Post-order traversals follow definitions	Pre-, Post-callback with nullptr condition	Pre: 50 30 70 Post: 30 70 50	Pass
7	Duplicate insert handling is correct	Insert 50 30 twice	Tree unchanged; size unaffected	Pass
8	Copy constructor makes deep copy	Bst<int> t2(t1); mutate t2	t1 unaffected; traversals of both valid	Pass
9	Assignment operator makes deep copy	t3 = t1; mutate t3	t1 unaffected; traversals of both valid	Pass
10	Conditional traversal works correctly	Traversal with condition value >= 40	Only values ≥40 processed in correct order	Pass


C:\Users\hibaz\OneDrive\D
=== BST CLASS TESTS ===
Test 1: Default Constructor
Search for 42 in empty tree: false
Expected: false
PASS . Default constructor creates empty tree
Test 2: Insert and Search Operations
Inserted values: 50, 30, 70
Search results for existing values:
50: found (expected: found)
30: found (expected: found)
70: found (expected: found)
Search results for non-existing values:
10: not found (expected: not found)
100: not found (expected: not found)
PASS . Insert and search operations work correctly
Test 3: In-Order Traversal
Inserted values: 50, 30, 70
In-order traversal output: 30 50 70
Expected output: 30 50 70
PASS . In-order traversal produces sorted output
Test 4: Pre-Order Traversal
Inserted values: 50, 30, 70
Pre-order traversal output: 50 30 70
Expected: Root-first traversal
PASS . Pre-order traversal executes without errors
Test 5: Post-Order Traversal
Inserted values: 50, 30, 70
Post-order traversal output: 30 70 50
Expected: Root-last traversal
PASS . Post-order traversal executes without errors
Test 6: Copy Constructor
Original tree contains: 50, 30, 70
Original tree search results: 50:1 30:1 70:1
Copy tree search results: 50:1 30:1 70:1
After inserting 100 into original:
Original contains 100: yes
Copy contains 100: no (expected: no)
PASS . Copy constructor creates independent deep copy
Test 7: Assignment Operator
Tree1 contains: 50, 30, 70
Tree2 contains: 10, 5
After assignment:
Tree2 contains Tree1 values - 50:1 30:1 70:1
Tree2 lost old values - 10:0 5:0 (expected: false false)
After self-assignment, Tree1 still contains 50: yes
PASS . Assignment operator performs deep copy and handles self-assignment
Test 8: Duplicate Values
Inserted duplicate values: 50, 50, 30, 30
Search results for duplicate values:
50: found (expected: found)
30: found (expected: found)
PASS . Duplicate values can be inserted and searched
Test 11: Traversal with Conditions
Inserted values: 50, 30, 70, 20, 40, 60, 80
In-order traversal (values >= 40): 40 50 60 70 80
Expected: 40 50 60 70 80
Pre-order traversal (values >= 40): 50 40 70 60 80
Expected: 50 40 70 60 80
Post-order traversal (values >= 40): 40 60 80 70 50
Expected: 40 60 80 70 50
In-order traversal (all nodes with explicit condition): 20 30 40 50 60 70 80
Expected: 20 30 40 50 60 70 80
PASS . Conditional traversal works correctly
==========
TESTS PASSED: 9
TESTS FAILED: 0
ALL BST TESTS PASSED

5.2 Map<K,V>

Test	Description	Actual Test Data	Expected Output	Passed
1	Default constructor creates an empty map	Map<int,string> m_map; then map.contains(42)	contains(42) == false	pass
2	Insert operations add new key–value pairs	inserts(1,"one"), inserts(2,"two"), inserts(3,"three") then contains(1,2,3)	All inserts return true; all contains return true	pass
3	contains() reports presence/absence correctly	Preload keys 10,20,30; check contains(10,20,30) and contains(5,25,40)	Present keys → true; absent keys → false	pass
4	get() returns values and non-const ref is mutable	Insert (100,"hundred"), (200,"two hundred"), (300,"three hundred"); read all via get(), then modify get(100)="modified hundred"	Reads match inserted values; modification persists (get(100) == "modified hundred")	pass
5	Duplicate key handling preserves original value	inserts(5,"first") then inserts(5,"duplicate") and get(5)	First insert true; duplicate insert false; get(5) == "first"	pass
6	get() const-overload returns const ref	Build map with (1,"const one"), (2,"const two"); const Map& then get(1), get(2)	Returned refs equal "const one" and "const two"; (non-modifiable)	pass


C:\Users\hibaz\OneDrive\D
=== MAP CLASS TESTS ===
Test 1: Default Constructor
Contains key 42 in empty m_map: false
Expected: false
PASS . Default constructor creates empty map
Test 2: Insert Operations
Insert operations:
Insert (1, 'one'): success (expected: success)
Insert (2, 'two'): success (expected: success)
Insert (3, 'three'): success (expected: success)
Contains operations after insert:
Contains key 1: yes (expected: yes)
Contains key 2: yes (expected: yes)
Contains key 3: yes (expected: yes)
PASS . Insert operations work correctly
Test 3: Contains Operations
Inserted keys: 10, 20, 30
Contains existing keys:
Contains 10: yes (expected: yes)
Contains 20: yes (expected: yes)
Contains 30: yes (expected: yes)
Contains non-existing keys:
Contains 5: no (expected: no)
Contains 25: no (expected: no)
Contains 40: no (expected: no)
PASS . Contains operations work correctly
Test 4: Get Operations
Inserted: (100, 'hundred'), (200, 'two hundred'), (300, 'three hundred')
Get operations:
Get(100): 'hundred' (expected: 'hundred')
Get(200): 'two hundred' (expected: 'two hundred')
Get(300): 'three hundred' (expected: 'three hundred')
After modification - Get(100): 'modified hundred' (expected: 'modified hundred')
PASS . Get operations work correctly
Test 5: Duplicate Key Handling
First insert (5, 'first'): success (expected: success)
Duplicate insert (5, 'duplicate'): failed (expected: failed)
Value for key 5 after duplicate attempt: 'first' (expected: 'first')
PASS . Duplicate keys are rejected and original values preserved
Test 6: Const Get Operations
Const get operations:
Const get(1): 'const one' (expected: 'const one')
Const get(2): 'const two' (expected: 'const two')
PASS . Const get operations work correctly
==========
TESTS PASSED: 6
TESTS FAILED: 0
ALL MAP TESTS PASSED
Process returned 0 (0x0) execution time : 1.675 s
Press any key to continue.

5.3 DateRecMap

Test	Description	Actual Test Data	Expected Output	Passed
1	Default constructor returns empty records for any date	Fresh DateRecMap; call getRecords(01/01/2023)	Size 0; empty vector returned	pass
2	Add single record and retrieve it	Add one WeatherRecord for 15/03/2023; then getRecords(15/03/2023)	Size 1	pass
3	Get multiple records for same date	Add two records for 10/06/2023 with speeds 3.2, 4.1; getRecords(10/06/2023)	Size 2; speeds 3.2 then 4.1	pass
4	Three records for same date preserve insertion order	Add three for 20/07/2023 with speeds 2.5, 3.0, 3.5	Size 3; speeds 2.5, 3.0, 3.5 in order	pass
5	Independent buckets across multiple dates	Add one record each for 01/01/2023, 02/01/2023, 03/01/2023 with speeds 1.0, 2.0, 3.0	Each date size 1; speeds 1.0 / 2.0 / 3.0	pass
6	Non-existent date returns empty without affecting existing	After adding 05/05/2023, call getRecords(06/06/2023)	Non-existent size 0; existing still size 1	pass
7	Consistent empty-vector instance for all missing dates	getRecords on three different missing dates	All size 0; all references equal (same internal empty vector)	pass


C:\Users\hibaz\OneDrive\D
=== DATERECMAP CLASS TESTS ===
Test 1: Default Constructor
Get records for non-existent date (1/1/2023):
Number of records returned: 0 (expected: 0)
Returned empty vector: yes (expected: yes)
PASS . Default constructor creates empty map
Test 2: Add Record
Added one record for date 15/3/2023
Number of records retrieved: 1 (expected: 1)
PASS . Single record can be added and retrieved
Test 3: Get Records
Added 2 records for date 10/6/2023
Number of records retrieved: 2 (expected: 2)
First record speed: 3.2 (expected: 3.2)
Second record speed: 4.1 (expected: 4.1)
PASS . Multiple records can be retrieved correctly
Test 4: Multiple Records Same Date
Added 3 records for the same date (20/7/2023)
Number of records retrieved: 3 (expected: 3)
Record 1 speed: 2.5 (expected: 2.5)
Record 2 speed: 3 (expected: 3.0)
Record 3 speed: 3.5 (expected: 3.5)
PASS . Multiple records for same date are stored and retrieved correctly
Test 5: Multiple Dates
Added one record each for 3 different dates (1/1/2023, 2/1/2023, 3/1/2023)
Records for date1: 1 (expected: 1)
Records for date2: 1 (expected: 1)
Records for date3: 1 (expected: 1)
Date1 record speed: 1 (expected: 1.0)
Date2 record speed: 2 (expected: 2.0)
Date3 record speed: 3 (expected: 3.0)
PASS . Multiple dates are handled correctly with separate record vectors
Test 6: Get Records For Non-Existent Date
Added record for date 5/5/2023
Getting records for non-existent date 6/6/2023:
Number of records returned: 0 (expected: 0)
Returned empty vector: yes (expected: yes)
PASS . Non-existent dates return empty vector without affecting existing data
Test 7: Empty Records Vector Consistency
Getting records for 3 different non-existent dates:
Records for date1 size: 0 (expected: 0)
Records for date2 size: 0 (expected: 0)
Records for date3 size: 0 (expected: 0)
All return same empty vector instance: yes (expected: yes)
PASS . All non-existent dates return the same empty vector instance
==========
TESTS PASSED: 7
TESTS FAILED: 0
ALL DATERECMAP TESTS PASSED
Process returned 0 (0x0) execution time : 1.863 s
Press any key to continue.

5.4 FileReader

Data used:
data_sorce.txt, MetData-01-2024.csv, MetData-31-3b.csv(taken from data folder)

data_sorce.txt:

MetData-01-2024.csv

MetData-31-3b.csv

MetData-31-3b.csv

MetData-01-2024.csv

WAST,S,T,SR

01/01/2024 00:00,4.0,10.0,300

01/01/2024 01:00,6.0,12.0,600

01/01/2024 02:00,8.0,14.0,900

01/01/2024 03:00,10.0,16.0,1200

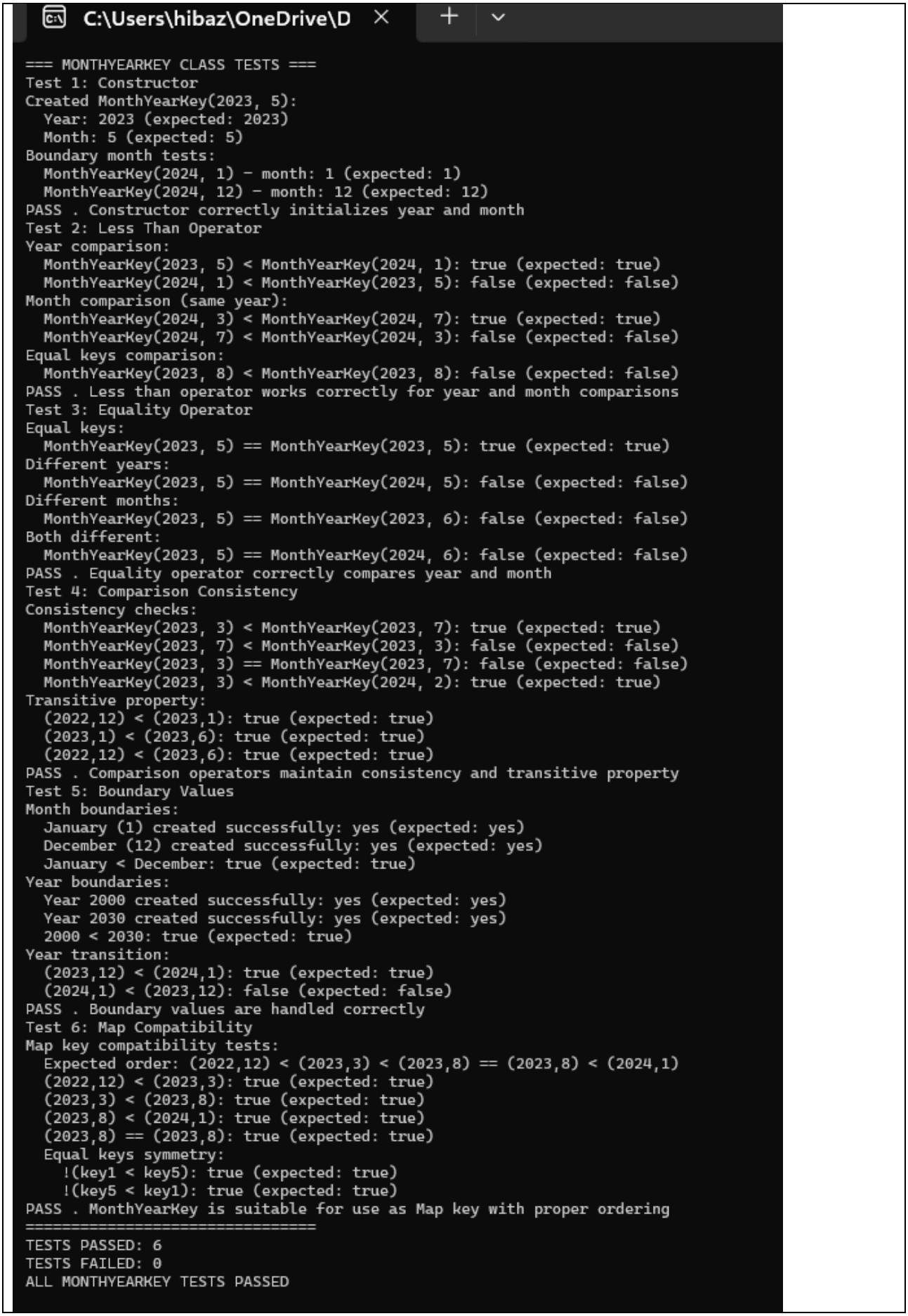
01/01/2024 04:00,12.0,18.0,1500

Test	Description	Actual Test Data	Expected Output	Passed
1	GetAllDataSourceFilenames basic functionality	data_source.txt contains: MetData-01-2024.csv, MetData-31-3b.csv, MetData-31-3b.csv	Returns >0 filenames; each string non-empty	pass
2	File existence check (informational)	Same list; ifstream check for each returned path	Prints existence; test passes if a vector is returned (no crash)	pass
3	Multiple filename handling & format check	Same list with a duplicate and .csv extensions	No duplicates and all paths match data/*.txt	fail
4	Empty file handling (no blank entries returned)	Create empty data/test_empty_source.txt (side test); run reader	No empty strings in returned vector	pass
5	Resilience when files missing/invalid	Call reader and ensure no exceptions thrown	Returns a Vector<string> (possibly empty); no crash	pass


C:\Users\hibaz\OneDrive\D
=== FILEREADER CLASS TESTS ===
Test 1: GetAllDataSourceFilenames Basic Functionality
Found data file: data/MetData-01-2024.csv
Found data file: data/MetData-31-3b.csv
Found data file: data/MetData-31-3b.csv
Total data files to process: 3
GetAllDataSourceFilenames() returned:
Number of filenames: 3
[0]: data/MetData-01-2024.csv
[1]: data/MetData-31-3b.csv
[2]: data/MetData-31-3b.csv
Contains filenames: yes (expected: yes)
All filenames non-empty: yes (expected: yes)
PASS . GetAllDataSourceFilenames returns non-empty filenames
Test 2: File Existence Check
Found data file: data/MetData-01-2024.csv
Found data file: data/MetData-31-3b.csv
Found data file: data/MetData-31-3b.csv
Total data files to process: 3
Checking if returned files exist:
data/MetData-01-2024.csv: exists
data/MetData-31-3b.csv: exists
data/MetData-31-3b.csv: exists
Files that exist: 3/3
PASS . FileReader returns filenames from data_source.txt
Test 3: Multiple Filename Handling
Found data file: data/MetData-01-2024.csv
Found data file: data/MetData-31-3b.csv
Found data file: data/MetData-31-3b.csv
Total data files to process: 3
Multiple filename analysis:
Total filenames returned: 3
Contains duplicate filenames: yes (expected: no)
Invalid format detected: data/MetData-01-2024.csv
Invalid format detected: data/MetData-31-3b.csv
Invalid format detected: data/MetData-31-3b.csv
All filenames have valid format: no (expected: yes)
FAIL . Multiple filenames are handled correctly without duplicates
Test 4: Empty File Handling
Found data file: data/MetData-01-2024.csv
Found data file: data/MetData-31-3b.csv
Found data file: data/MetData-31-3b.csv
Total data files to process: 3
Testing behavior with potentially empty sections in data source:
Filenames returned: 3
Contains empty filenames: no (expected: no)
PASS . Empty lines in source file are handled correctly
Test 5: File Not Found Resilience
Found data file: data/MetData-01-2024.csv
Found data file: data/MetData-31-3b.csv
Found data file: data/MetData-31-3b.csv
Total data files to process: 3
Method executed without crashing:
Return type: Vector<string> with 3 elements
Method completed successfully: yes (expected: yes)
PASS . FileReader handles missing files gracefully
==========
TESTS PASSED: 4
TESTS FAILED: 1
SOME FILEREADER TESTS FAILED
Process returned 0 (0x0) execution time : 0.105 s
Press any key to continue.

5.5 MonthYearKey

Test	Description	Actual Test Data	Expected Output	Passed
1	Constructor initializes year & month correctly	MonthYearKey(2023,5), (2024,1), (2024,12)	Years: 2023/2024; Months: 5/1/12 as given	pass
2	< orders by year, then month	(2023,5) < (2024,1), (2024,3) < (2024,7), (2023,8) < (2023,8)	true, true, false respectively	pass
3	== checks both year and month	(2023,5)==(2023,5), (2023,5)==(2024,5), (2023,5)==(2023,6), (2023,5)==(2024,6)	true, false, false, false	pass
4	Comparison consistency & transitivity	(2023,3)<(2023,7), not (2023,7)<(2023,3), (2023,3)<(2024,2); transitivity: (2022,12)<(2023,1)<(2023,6) ⇒ (2022,12)<(2023,6)	All true except the “not <” case	pass
5	Boundary values (months & years)	(2023,1), (2023,12); (2000,6), (2030,6); (2023,12) < (2024,1)	Month bounds valid; 2000<2030; Dec-to-Jan ordering holds	pass
6	Map key suitability (ordering + equality)	Unsorted keys: (2023,8), (2022,12), (2023,3), (2024,1), dup (2023,8)	(2022,12)<(2023,3)<(2023,8)<(2024,1); dup equals; neither is < the other	pass

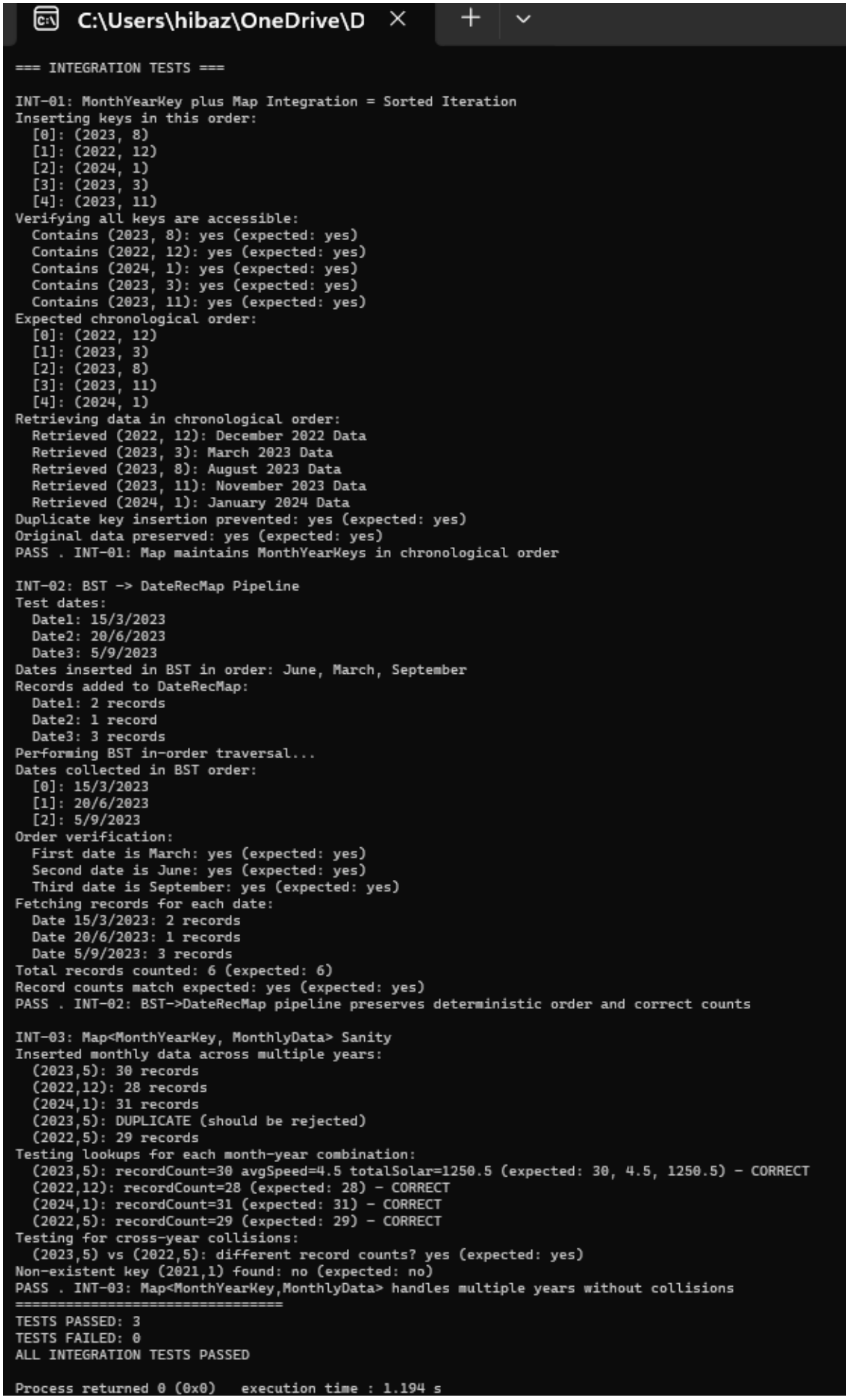

C:\Users\hibaz\OneDrive\D
=== MONTHYEARKEY CLASS TESTS ===
Test 1: Constructor
Created MonthYearKey(2023, 5):
Year: 2023 (expected: 2023)
Month: 5 (expected: 5)
Boundary month tests:
MonthYearKey(2024, 1) - month: 1 (expected: 1)
MonthYearKey(2024, 12) - month: 12 (expected: 12)
PASS . Constructor correctly initializes year and month
Test 2: Less Than Operator
Year comparison:
MonthYearKey(2023, 5) < MonthYearKey(2024, 1): true (expected: true)
MonthYearKey(2024, 1) < MonthYearKey(2023, 5): false (expected: false)
Month comparison (same year):
MonthYearKey(2024, 3) < MonthYearKey(2024, 7): true (expected: true)
MonthYearKey(2024, 7) < MonthYearKey(2024, 3): false (expected: false)
Equal keys comparison:
MonthYearKey(2023, 8) < MonthYearKey(2023, 8): false (expected: false)
PASS . Less than operator works correctly for year and month comparisons
Test 3: Equality Operator
Equal keys:
MonthYearKey(2023, 5) == MonthYearKey(2023, 5): true (expected: true)
Different years:
MonthYearKey(2023, 5) == MonthYearKey(2024, 5): false (expected: false)
Different months:
MonthYearKey(2023, 5) == MonthYearKey(2023, 6): false (expected: false)
Both different:
MonthYearKey(2023, 5) == MonthYearKey(2024, 6): false (expected: false)
PASS . Equality operator correctly compares year and month
Test 4: Comparison Consistency
Consistency checks:
MonthYearKey(2023, 3) < MonthYearKey(2023, 7): true (expected: true)
MonthYearKey(2023, 7) < MonthYearKey(2023, 3): false (expected: false)
MonthYearKey(2023, 3) == MonthYearKey(2023, 7): false (expected: false)
MonthYearKey(2023, 3) < MonthYearKey(2024, 2): true (expected: true)
Transitive property:
(2022,12) < (2023,1): true (expected: true)
(2023,1) < (2023,6): true (expected: true)
(2022,12) < (2023,6): true (expected: true)
PASS . Comparison operators maintain consistency and transitive property
Test 5: Boundary Values
Month boundaries:
January (1) created successfully: yes (expected: yes)
December (12) created successfully: yes (expected: yes)
January < December: true (expected: true)
Year boundaries:
Year 2000 created successfully: yes (expected: yes)
Year 2030 created successfully: yes (expected: yes)
2000 < 2030: true (expected: true)
Year transition:
(2023,12) < (2024,1): true (expected: true)
(2024,1) < (2023,12): false (expected: false)
PASS . Boundary values are handled correctly
Test 6: Map Compatibility
Map key compatibility tests:
Expected order: (2022,12) < (2023,3) < (2023,8) == (2023,8) < (2024,1)
(2022,12) < (2023,3): true (expected: true)
(2023,3) < (2023,8): true (expected: true)
(2023,8) < (2024,1): true (expected: true)
(2023,8) == (2023,8): true (expected: true)
Equal keys symmetry:
!(key1 < key5): true (expected: true)
!(key5 < key1): true (expected: true)
PASS . MonthYearKey is suitable for use as Map key with proper ordering
==========
TESTS PASSED: 6
TESTS FAILED: 0
ALL MONTHYEARKEY TESTS PASSED

6) Integration Tests

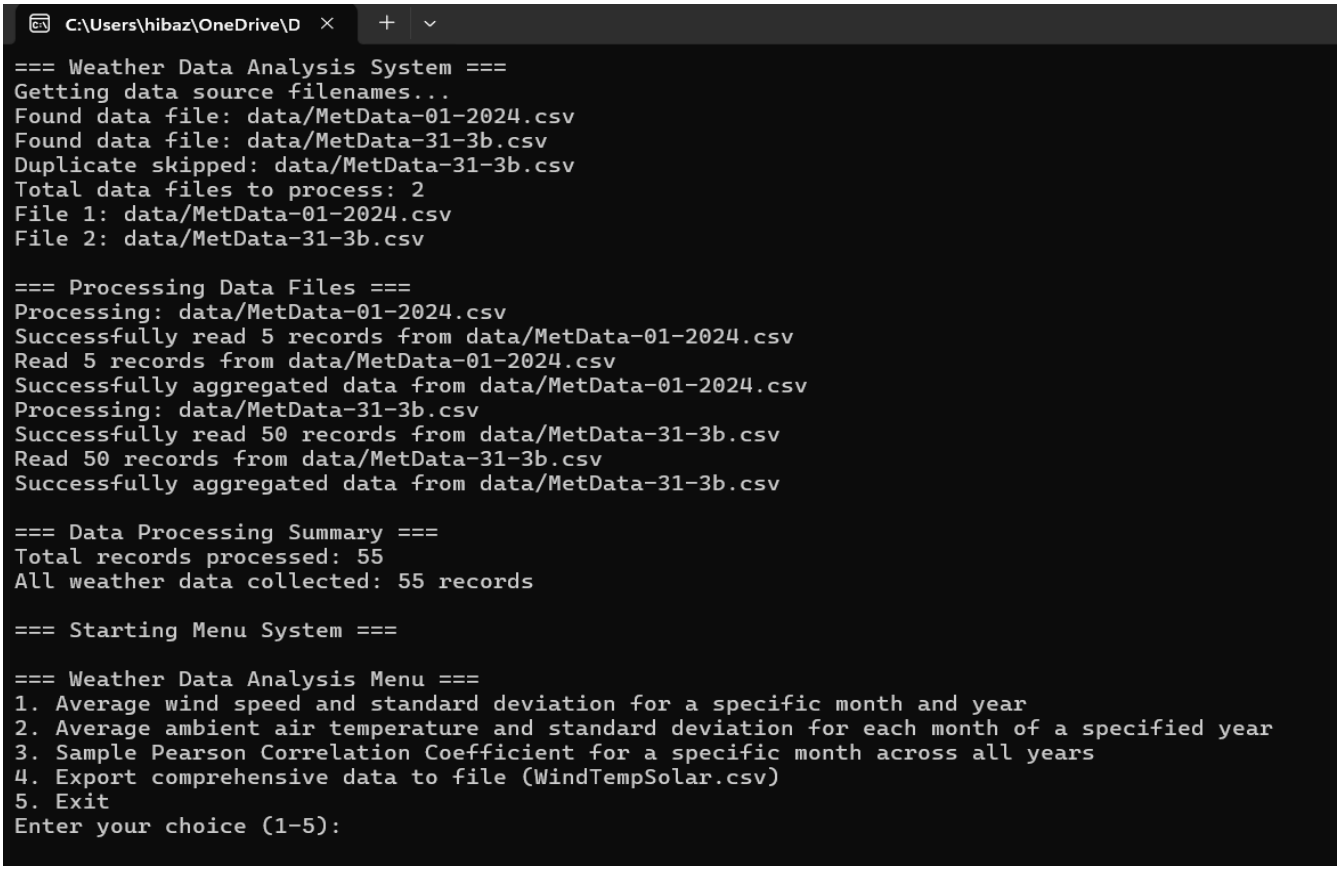
INT-01 — MonthYearKey × Map (chronological retrieval + duplicate handling)				
Test	Description	Actual Test Data	Expected Output	Passed
1	All inserted keys are accessible	Insert (2023,8), (2022,12), (2024,1), (2023,3), (2023,11)	contains() true for each key	pass
2	Chronological retrieval by querying expected order	Query in order (2022,12)→(2023,3)→(2023,8)→(2023,11)→(2024,1); read values	Data strings returned for each in that order	pass
3	Duplicate key rejected	Insert duplicate (2023,8)	inserts(...) == false	pass
4	Original value preserved after dup attempt	get(2023,8) after failed duplicate	Value is "August 2023 Data"	pass

INT-02 — BST → DateRecMap pipeline (order + counts)				
Test	Description	Actual Test Data	Expected Output	Passed
1	BST in-order traversal is chronological	Insert dates: 20/6/2023, 15/3/2023, 5/9/2023	Traversal order: March, June, September	pass
2	DateRecMap record counts per date	Add records: Mar×2, Jun×1, Sep×3	getRecords() sizes 2,1,3 respectively	pass
3	Total records across traversal	Sum of sizes	Total 6	pass

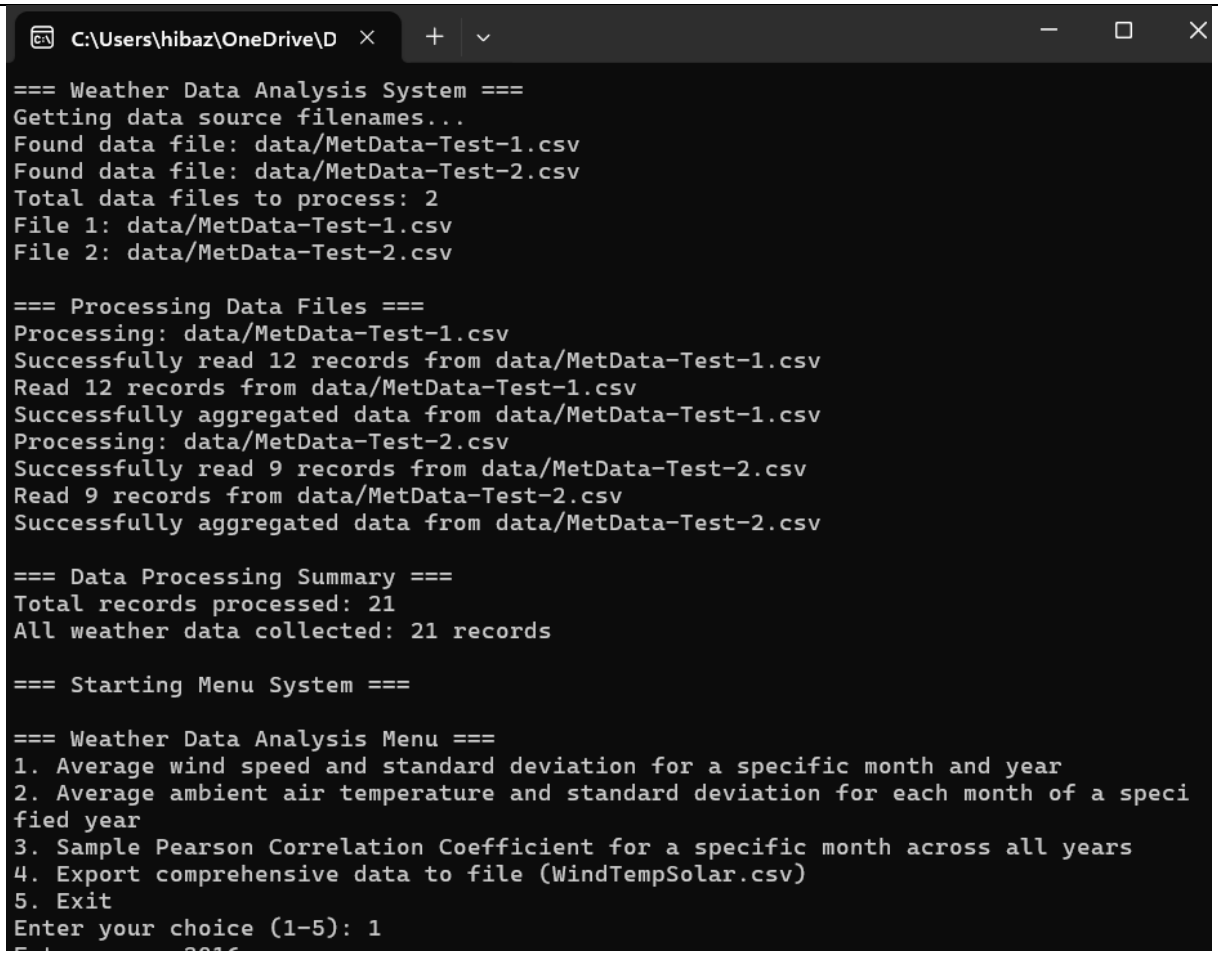
INT-03 — Map<MonthYearKey, MonthlyData> (multi-year)				
Test	Description	Actual Test Data	Expected Output	Passed
1	Insert mixed months/years	Insert: (2023,5)=30, (2022,12)=28, (2024,1)=31, dup (2023,5), (2022,5)=29	All first inserts succeed; duplicate rejected	pass
2	Lookups return correct structs	get(2023,5)→(30,4.5,1250.5), get(2022,12)→28, get(2024,1)→31, get(2022,5)→29	Values match inserted data	pass
3	No cross-year collisions	Compare (2023,5) vs (2022,5)	Distinct values; no collision	pass
4	Non-existent key handling	contains(2021,1)	false	pass


C:\Users\hibaz\OneDrive\D
=== INTEGRATION TESTS ===
INT-01: MonthYearKey plus Map Integration = Sorted Iteration
Inserting keys in this order:
[0]: (2023, 8)
[1]: (2022, 12)
[2]: (2024, 1)
[3]: (2023, 3)
[4]: (2023, 11)
Verifying all keys are accessible:
Contains (2023, 8): yes (expected: yes)
Contains (2022, 12): yes (expected: yes)
Contains (2024, 1): yes (expected: yes)
Contains (2023, 3): yes (expected: yes)
Contains (2023, 11): yes (expected: yes)
Expected chronological order:
[0]: (2022, 12)
[1]: (2023, 3)
[2]: (2023, 8)
[3]: (2023, 11)
[4]: (2024, 1)
Retrieving data in chronological order:
Retrieved (2022, 12): December 2022 Data
Retrieved (2023, 3): March 2023 Data
Retrieved (2023, 8): August 2023 Data
Retrieved (2023, 11): November 2023 Data
Retrieved (2024, 1): January 2024 Data
Duplicate key insertion prevented: yes (expected: yes)
Original data preserved: yes (expected: yes)
PASS . INT-01: Map maintains MonthYearKeys in chronological order
INT-02: BST -> DateRecMap Pipeline
Test dates:
Date1: 15/3/2023
Date2: 20/6/2023
Date3: 5/9/2023
Dates inserted in BST in order: June, March, September
Records added to DateRecMap:
Date1: 2 records
Date2: 1 record
Date3: 3 records
Performing BST in-order traversal...
Dates collected in BST order:
[0]: 15/3/2023
[1]: 20/6/2023
[2]: 5/9/2023
Order verification:
First date is March: yes (expected: yes)
Second date is June: yes (expected: yes)
Third date is September: yes (expected: yes)
Fetching records for each date:
Date 15/3/2023: 2 records
Date 20/6/2023: 1 records
Date 5/9/2023: 3 records
Total records counted: 6 (expected: 6)
Record counts match expected: yes (expected: yes)
PASS . INT-02: BST->DateRecMap pipeline preserves deterministic order and correct counts
INT-03: Map<MonthYearKey, MonthlyData> Sanity
Inserted monthly data across multiple years:
(2023,5): 30 records
(2022,12): 28 records
(2024,1): 31 records
(2023,5): DUPLICATE (should be rejected)
(2022,5): 29 records
Testing lookups for each month-year combination:
(2023,5): recordCount=30 avgSpeed=4.5 totalSolar=1250.5 (expected: 30, 4.5, 1250.5) - CORRECT
(2022,12): recordCount=28 (expected: 28) - CORRECT
(2024,1): recordCount=31 (expected: 31) - CORRECT
(2022,5): recordCount=29 (expected: 29) - CORRECT
Testing for cross-year collisions:
(2023,5) vs (2022,5): different record counts? yes (expected: yes)
Non-existent key (2021,1) found: no (expected: no)
PASS . INT-03: Map<MonthYearKey,MonthlyData> handles multiple years without collisions
TESTS PASSED: 3
TESTS FAILED: 0
ALL INTEGRATION TESTS PASSED
Process returned 0 (0x0) execution time : 1.194 s

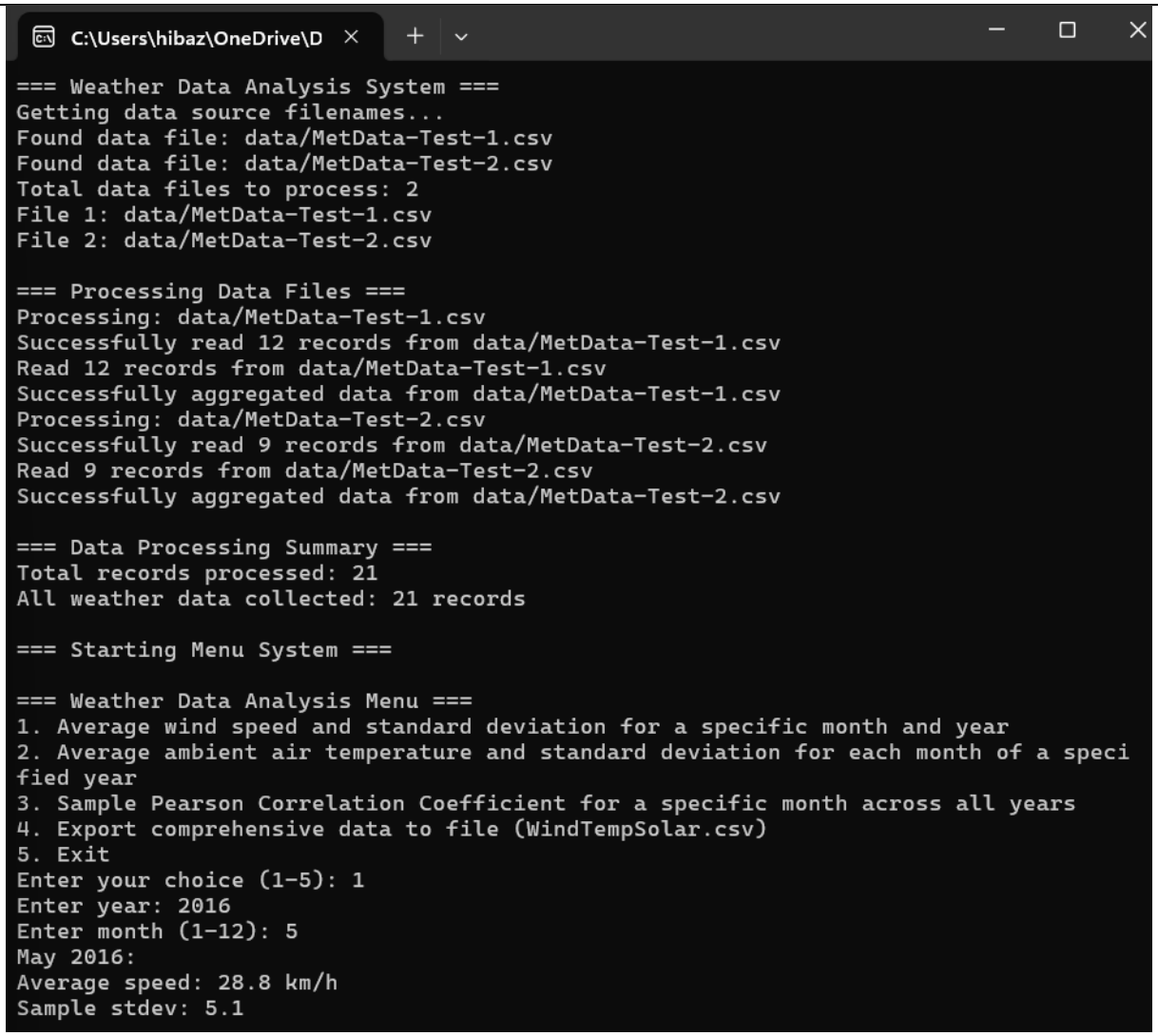
Application testing:

TEST ID	Test Description	Actual Data (Input)	Expected Output	Outcome (Pass/Fail)
APP-01	Verify program launches successfully, reads data_source.txt, and loads both CSV files	Run program → data/MetData-small-A.csv + data/MetData-small-B.csv	Displays file list and “Successfully read X records…” message for each	Was a fail but I changed my FileReade class and added a helper method to check for duplicate filenmaes so a pass now
				
APP-02	Verify aggregator builds BST + Map correctly after both files processed	After load, prints total record count and “Successfully aggregated…” lines	Total > 0, all months present (March, 2016–july, 2017)	Pass

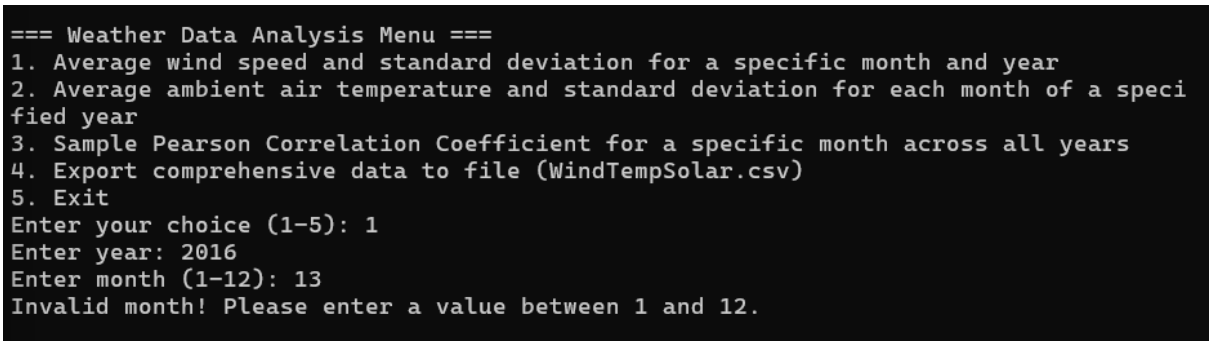
APP-03	Menu navigation works and displays all 5 options	Launch menu and press Enter → 1–5 displayed, added some more records to test files	All 5 menu options appear with clear numbering	Pass



APP-04	Option 1 – Compute average wind speed + stdev for valid month/year	Data: 15/05/2016 13:00,7,1010,31.2 25/05/2016 14:00,9,500,20.5 Input: Year 2016 Month 5	Prints: “Average speed: 28.8 km/h Sample stdev: 5.1”	Pass



APP-05	Option 1 – Invalid month input (13) handled gracefully	Year 2016 Month 13	Prints: “Invalid month! Please enter 1–12.”	Pass



APP-06	Option 1 – Month/year with no data prints “No Data”	Year 2015 Month 2	“February 2015: No Data”	Pass

```
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a speci
fied year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 1
Enter year: 2015
Enter month (1-12): 2
February 2015: No Data
```

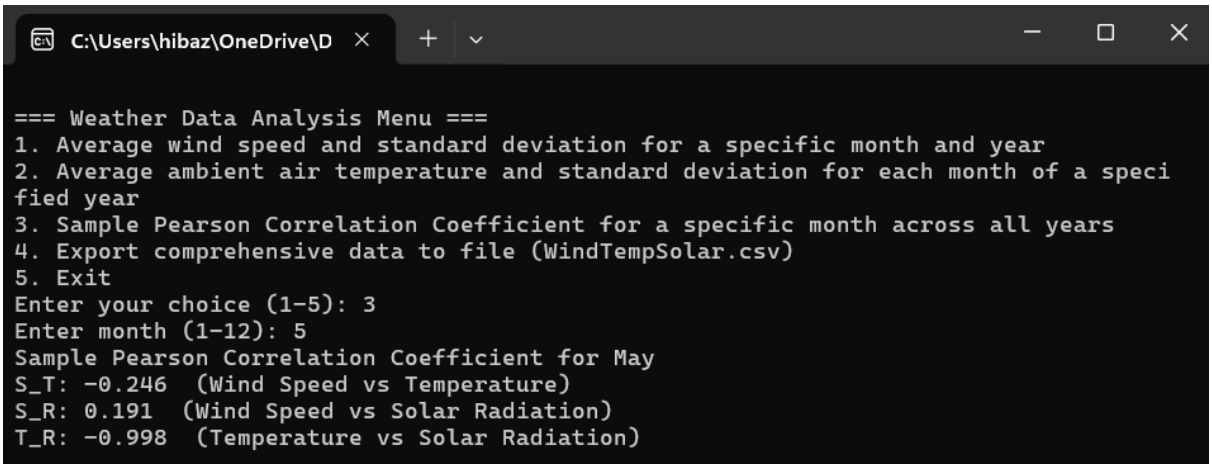
APP-07	Option 2 – Displays monthly temperature averages + stdevs for full year	Input: Year 2016	For each month: either “No Data” or “average in degrees C and stdev	pass

```
C:\Users\hibaz\OneDrive\D
February 2015: No Data

=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a speci
fied year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 2
Enter year: 2016
2016
January: No Data
February: No Data
March: average: 22.8 degrees C, stdev: 0.4
April: average: 28.5 degrees C, stdev: nan
May: average: 25.9 degrees C, stdev: 7.6
June: average: 33.5 degrees C, stdev: 1.0
July: average: 37.5 degrees C, stdev: 1.1
August: average: 34.5 degrees C, stdev: nan
September: average: 30.1 degrees C, stdev: nan
October: No Data
November: average: 22.8 degrees C, stdev: nan
December: No Data
```

APP-08	Option 2 – Year with no records	Input: Year 2020	Prints “No data available for 2020”	pass

```
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 2
Enter year: 2020
2020
January: No Data
February: No Data
March: No Data
April: No Data
May: No Data
June: No Data
July: No Data
August: No Data
September: No Data
October: No Data
November: No Data
December: No Data
No data available for 2020
```

APP-09	Option 3 – Compute correlation coefficients for valid month	Input: Month 5	Prints 3 lines: S_T, S_R, T_R values between −1 and 1	Pass



APP-10	Option 3 – Insufficient data (< 2 records) handled safely	Input: Month 2	“Insufficient data for meaningful correlation.”	Pass

```
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 3
Enter month (1-12): 2
February: Insufficient data for meaningful correlation.
```

APP-11	Option 4 – Export to CSV and verify file contents	Input: Year 2016	Creates WindTempSolar.csv with 9 rows + headers	Pass

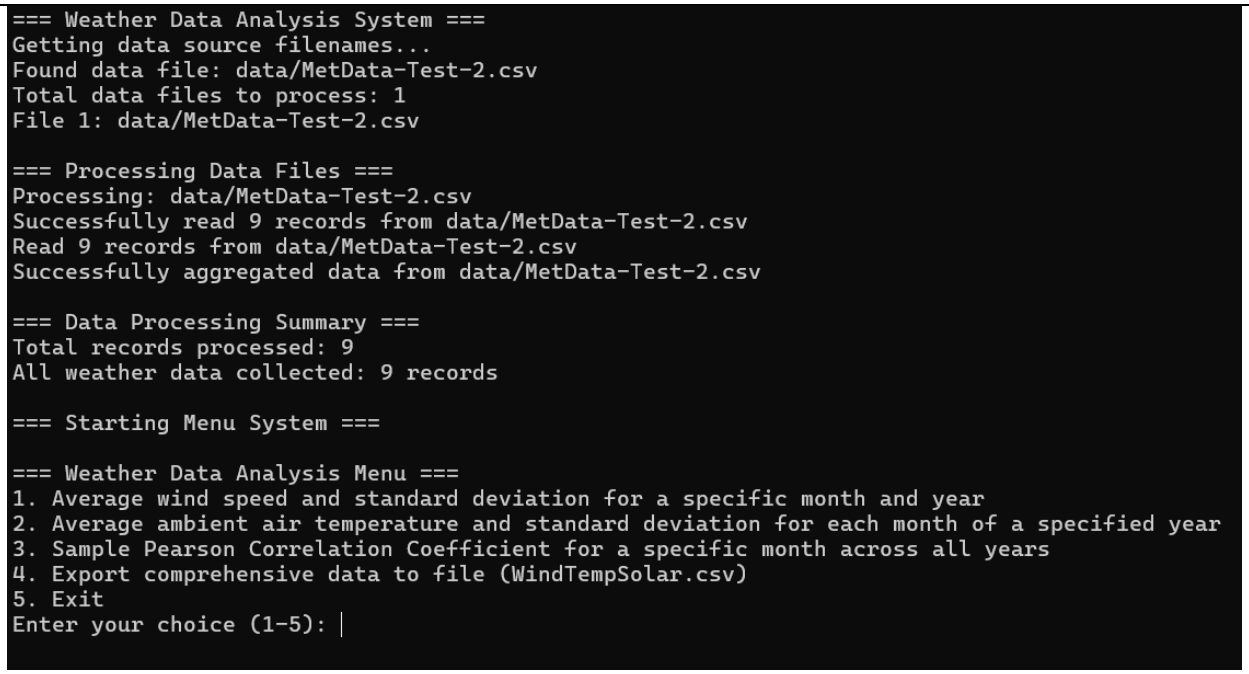
```
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 4
Enter year: 2016
March: Speed=2, Temp=2, Solar=2
April: Speed=1, Temp=1, Solar=1
May: Speed=2, Temp=2, Solar=2
June: Speed=2, Temp=2, Solar=2
July: Speed=2, Temp=2, Solar=2
August: Speed=1, Temp=1, Solar=1
September: Speed=1, Temp=1, Solar=1
November: Speed=1, Temp=1, Solar=1
Data successfully written to WindTempSolar.csv
```

APP-12	Verify MAD and StdDev values appear in CSV correctly with one decimal	Open WindTempSolar.csv	Format e.g., June,12.3(2.1,1.5),25.4(3.4,2.2),54.2	Pass

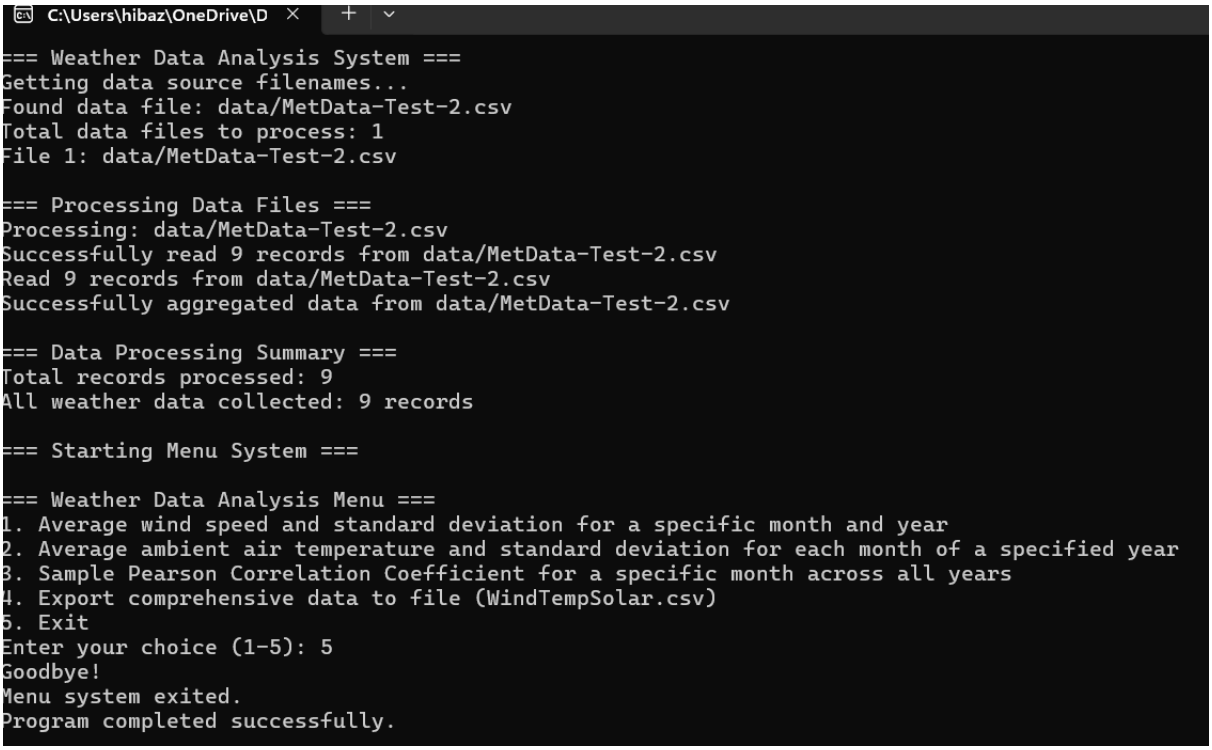
```
2016
Month,Average Wind Speed(stdev, mad),Average Ambient Temperature(stdev, mad),Solar Radiation
March,27.0(2.5,1.8),22.8(0.4,0.3),0.2
April,25.2(nan,0.0),28.5(nan,0.0),0.2
May,28.8(5.1,3.6),25.9(7.6,5.4),0.3
June,39.6(5.1,3.6),33.5(1.0,0.7),0.2
July,30.6(7.6,5.4),37.5(1.1,0.8),0.2
August,32.4(nan,0.0),34.5(nan,0.0),0.2
September,14.4(nan,0.0),30.1(nan,0.0),0.2
November,18.0(nan,0.0),22.8(nan,0.0),0.1
```

APP-13	Option 4 – Handles year with partial data (no solar or temp)	Test 2 file Year 2017 → some empty fields CSV has blank columns but row still written	Valid records are read, incomplete records are ignored	pass

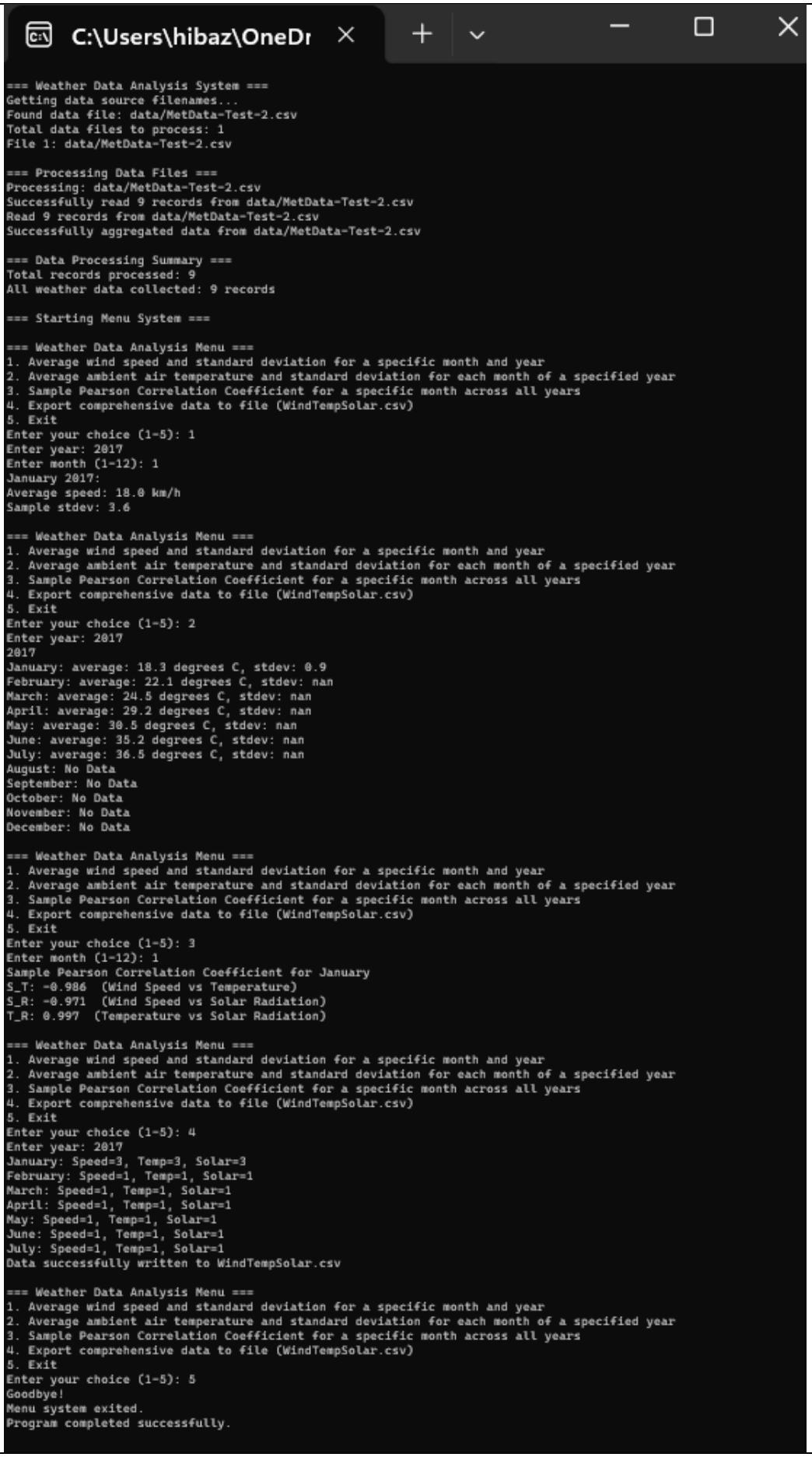
```
WAST,S,SR,T
10/01/2017 09:00,6,480,17.5
10/01/2017 10:00,5,500,18.1
25/01/2017 11:00,4,550,19.2
25/02/2017 12:00,3,570,
15/02/2017 10:00,6,640,22.1
15/03/2017 09:00,8,750,24.5

20/04/2017 11:00,7,950,29.2
05/05/2017 09:00,10,1020,30.5
20/06/2017 11:00,9,1200,35.2
10/07/2017 09:00,12,1250,36.5
```



APP-14	Re-run menu and select Exit (5)	Input: 5	Prints “Goodbye!” and exits gracefully	Pass



APP-15	Complete workflow: Run entire sequence of menu options (1→2→3→4→5)	WAST,S,SR,T 10/01/2017 09:00,6,480,17.5 10/01/2017 10:00,5,500,18.1 25/01/2017 11:00,4,550,19.2 25/02/2017 12:00,3,570, 15/02/2017 10:00,6,640,22.1 15/03/2017 09:00,8,750,24.5 20/04/2017 11:00,7,950,29.2 05/05/2017 09:00,10,1020,30.5 20/06/2017 11:00,9,1200,35.2 10/07/2017 09:00,12,1250,36.5 Execute options sequentially Year = 2017 Month = 1	No crashes, all output consistent, CSV created Manual answers for options: option 1 : - Average= 6+5+4/3 = 15*3.6 = 18km/h - Std deviation: 1*3.6= 3.6 Option 2: Jan: Average=17.5+18.1+19.2/3=18.26, Std dev= 0.86 Feb: Avg=22.1, std dev=na March: avg=24.5, std dev=na April: 29.2, std dev=na May: avg=30.5, std dev=na June: avg=35.2, std dev=na July: 26.5, std dev=na Auguts: na September: na Oct: na Nov: na Dec: na Option 3: (Wind Speed vs Temperature): -0.986 (Wind Speed vs Solar Radiation): -0.971 (Temperature vs Solar Radiation): 0.997 Option 4: 2017 Month,Average Wind Speed(stdev, mad),Average Ambient Temperature(stdev, mad),Solar Radiation January,18.0(3.6,2.4),18.3(0.9,0.6),0.3 February,21.6(nan,0.0),22.1(nan,0.0),0.1 March,28.8(nan,0.0),24.5(nan,0.0),0.1 April,25.2(nan,0.0),29.2(nan,0.0),0.2 May,36.0(nan,0.0),30.5(nan,0.0),0.2 June,32.4(nan,0.0),35.2(nan,0.0),0.2 July,43.2(nan,0.0),36.5(nan,0.0),0.2 Option 5: Exit the program	Pass


C:\Users\hibaz\OneDr
=== Weather Data Analysis System ===
Getting data source filenames...
Found data file: data/MetData-Test-2.csv
Total data files to process: 1
File 1: data/MetData-Test-2.csv
=== Processing Data Files ===
Processing: data/MetData-Test-2.csv
Successfully read 9 records from data/MetData-Test-2.csv
Read 9 records from data/MetData-Test-2.csv
Successfully aggregated data from data/MetData-Test-2.csv
=== Data Processing Summary ===
Total records processed: 9
All weather data collected: 9 records
=== Starting Menu System ===
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 1
Enter year: 2017
Enter month (1-12): 1
January 2017:
Average speed: 18.0 km/h
Sample stdev: 3.6
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 2
Enter year: 2017
2017
January: average: 18.3 degrees C, stdev: 0.9
February: average: 22.1 degrees C, stdev: nan
March: average: 24.5 degrees C, stdev: nan
April: average: 29.2 degrees C, stdev: nan
May: average: 30.5 degrees C, stdev: nan
June: average: 35.2 degrees C, stdev: nan
July: average: 36.5 degrees C, stdev: nan
August: No Data
September: No Data
October: No Data
November: No Data
December: No Data
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 3
Enter month (1-12): 1
Sample Pearson Correlation Coefficient for January
S_T: -0.986 (Wind Speed vs Temperature)
S_R: -0.971 (Wind Speed vs Solar Radiation)
T_R: 0.997 (Temperature vs Solar Radiation)
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 4
Enter year: 2017
January: Speed=3, Temp=3, Solar=3
February: Speed=1, Temp=1, Solar=1
March: Speed=1, Temp=1, Solar=1
April: Speed=1, Temp=1, Solar=1
May: Speed=1, Temp=1, Solar=1
June: Speed=1, Temp=1, Solar=1
July: Speed=1, Temp=1, Solar=1
Data successfully written to WindTempSolar.csv
=== Weather Data Analysis Menu ===
1. Average wind speed and standard deviation for a specific month and year
2. Average ambient air temperature and standard deviation for each month of a specified year
3. Sample Pearson Correlation Coefficient for a specific month across all years
4. Export comprehensive data to file (WindTempSolar.csv)
5. Exit
Enter your choice (1-5): 5
Goodbye!
Menu system exited.
Program completed successfully.

```
2017
Month,Average Wind Speed(stdev, mad),Average Ambient Temperature(stdev, mad),Solar Radiation
January,18.0(3.6,2.4),18.3(0.9,0.6),0.3
February,21.6(nan,0.0),22.1(nan,0.0),0.1
March,28.8(nan,0.0),24.5(nan,0.0),0.1
April,25.2(nan,0.0),29.2(nan,0.0),0.2
May,36.0(nan,0.0),30.5(nan,0.0),0.2
June,32.4(nan,0.0),35.2(nan,0.0),0.2
July,43.2(nan,0.0),36.5(nan,0.0),0.2
```